

OFFICIAL JOURNAL OF THE BRITISH HOROLOGICAL INSTITUTE

The Horological Journal



MARCH 2026
www.bhi.co.uk



A Clock Pendulum Timer

Designing a Simple Turret Clock Time Controller

Alan Wood



Introduction

My involvement with the village church clock began when I mentioned my interest to a member of the local community. The clock in question is a lovely ‘Cooke of York’ model, dated 1868, **Figure 1**. Having grown up local to York, it tugged at my heartstrings. It is in pristine condition, with a 1.25 s pendulum, a five-arm gravity escapement, and an hourly strike.

The village has rightly decided against automating the winding of the clock. This continues to be done manually three times per week by a dedicated group of volunteers. My involvement has become purely ‘tech support’, responding to any ‘clock has stopped’ call-outs and making timekeeping tweaks. The latter requires the addition or removal of pennies from the pendulum weight tray, with associated fine tweaks by ‘holding the fly’.

The Aim of the Project

At this point I should elaborate on what is ‘village time’. The villagers do not want their church clock to replace the BBC Greenwich Time Signal pips. They subconsciously absorb the striking hours as a rolling background reference to their busy lives. It is the constant that tells them it’s time to come in from the garden for a brew, take the dog for a walk, and take the children to and from school, etc – events that do not need, or demand, the last millisecond in accuracy. For them, it is a guide to time, not a definer of time. Besides, with sound travelling at around 1 ft per millisecond, everyone in the village will hear the clock striking at a slightly different time.

My aim has always been to try to keep the clock within plus or minus five seconds based on my penny tweaks and fly-based adjustments. A penny is specified as weighing 3.56 g. Providing weather temperature changes are gradual and winds are not high then the penny changes are relatively infrequent. Over a calendar year this generally amounts to a five- or six-penny variation.

Automating the ‘Pennies’

My curious engineering mind began to wonder if I could replace the weight tray pennies with something more controllable and with a defined automatic change profile. I was conscious that any solution and its installation would not have to damage the clock movement or enclosure, and be readily demountable.

Automatically raising and lowering weights on to the weight tray seemed a practical idea. My initial thought was to use a washbasin ball chain. This is available in different ball diameters. The problem with a ball chain is that it has a mind of its own. Should the chain miss the tray at any time, a cascade of the chain will occur as one ball, like a swarm of lemmings, follows another over the edge of the weight tray.

My solution was to use fishing weights clamped at regular intervals on to a fine nylon cord. I opted for 0.8 g lead shot (AAA). On this basis, four lead shots roughly equal one penny.



Figure 1. The clock is a ‘Cooke of York’ model, dated 1868.

The raising and lowering of the weights via the nylon cord would be controlled by a low-cost stepper motor. This, in turn, would be controlled by some ‘simple’ software code, running in an Arduino environment.

System Outline

Calculating the timing error was the first problem to be addressed. I needed to have an electronic signal (pulse) from the clock at the point of the hour strike, and be able to compare this pulse with a reference pulse from a time standard. Once compared, a software decision would be made to raise or lower weights via the stepper motor.

The source of this pair of timing pulses evolved with the project. The initial design used Hall-effect sensors. A customised DCF off-air clock movement was fitted with a small magnet on the minute hand, and this provided the ‘on the hour’ reference pulse. The strike timing pulse from the clock was also a Hall-effect sensor, triggered by a small magnet mounted on the bell striking mechanism.

The time difference between the two pulses was measured. Subject to the sign of the difference value, the processor would either drive the stepper clockwise (CW) or anti-clockwise (ACW) to raise or lower the dangling line of fishing weights. I will refer to this software logic as the Lead Lag Controller (LLC).

The sensor timing sources changed mid-project. The Hall-effect sensors were replaced with a GPS-based one pulse per hour reference standard in conjunction with a laser beam detection of the drop of the strike let-off lever (strike arm) at the hour. The change from Hall-effects sensors gave a more repeatable pair of sensor pulses.

The software for the LLC system was written in Arduino code and runs on a Sreed Xiao SAMD21 module. The stepper controller uses a second Xiao processor. To simplify development, these two functions were constructed on two separate PCB modules.

The Sreed Xiao was chosen as it has two interrupt inputs and two serial ports. The default port (Serial0) is used for programming and viewing the serial monitor activity via the Arduino IDE. The second serial (Serial1) is used to stream a packet of system monitoring data to external devices.

The Xiao processor in the LLC monitors the two sensor pulses. One from the 1pph GPS reference time standard and one from the clock strike lever. The software decides which pulse was present first and outputs a raise or lower signal to the second Xiao processor, which controls the stepper motor. The UP and DOWN signals to the stepper controller are 100ms fixed period pulses.

I am not a software guru, but can usually get by with cut and paste design using my accumulated knowledge from a number of Arduino projects. I rapidly discovered that on this project my dabble level 'cut and paste' software skills were not going to deliver a reliable solution.

AI Support

While discussing my plan for the project with a colleague, he suggested trying ChatGPT or Claude AI resources, something I had no experience of. These are AI code creation websites. In simple terms, you tell the AI what you are trying to achieve, what hardware you have available and it will very quickly deliver code that will satisfy the requirement.

It is not perfect. I used ChatGPT and found it is only as good as your ability to describe exactly what you want to happen, what hardware you are using, and what the interconnections are. The output result is only as good as the input data or, to take the inverse, garbage in will deliver garbage out.

It is likely that the initial code provided by ChatGPT will be 'not quite there'. It needs you as the client to try the code and clearly describe what is or isn't happening. This must be fed back so an iteration of the code can be created... and then you go round the loop again.

ChatGPT is an effective route for quickly getting working code without the drudgery of writing it by hand. My coding understanding has improved as a result of the experience. Anything I do not understand is patiently and simply explained by the AI resource and, refreshingly, without imparting any inferiority judgement.

I have found one aspect of the ChatGPT partnership that must be monitored and controlled – mission creep. There is a constant need to apply a handbrake to stop or at least slow the avalanche of code creep that can potentially ensue. Lots of 'it would be easy to add...' conversations crop up, which need to be firmly resisted until the core of the project is stable and running to plan.

LoRa Radio Link

Once the basic LLC was operational, I needed to be able to monitor the clock remotely at home. A remote monitoring system was implemented using an Arduino LoRa radio link. This allows remote monitoring and logging of the synchronising process.

The LoRa radio link operates within the 868MHz de-licensed frequency band. Once per hour, after comparing the timing of the two trigger pulses, the LLC sends a data packet to the LoRa transmit module via Serial(1) on the Xiao. The packet contains a whole raft of comma-delimited parameters relating to the synchronising process. The transmission is received in my workshop and the data is date stamped and displayed on a small OLED display, while also being stored on a SD memory card. The radio hardware consists of a radio transmitter and radio receiver PCB, and this adds two more Xiao processors to the bill of materials.

Resulting Changes after LoRa Monitoring Was Added

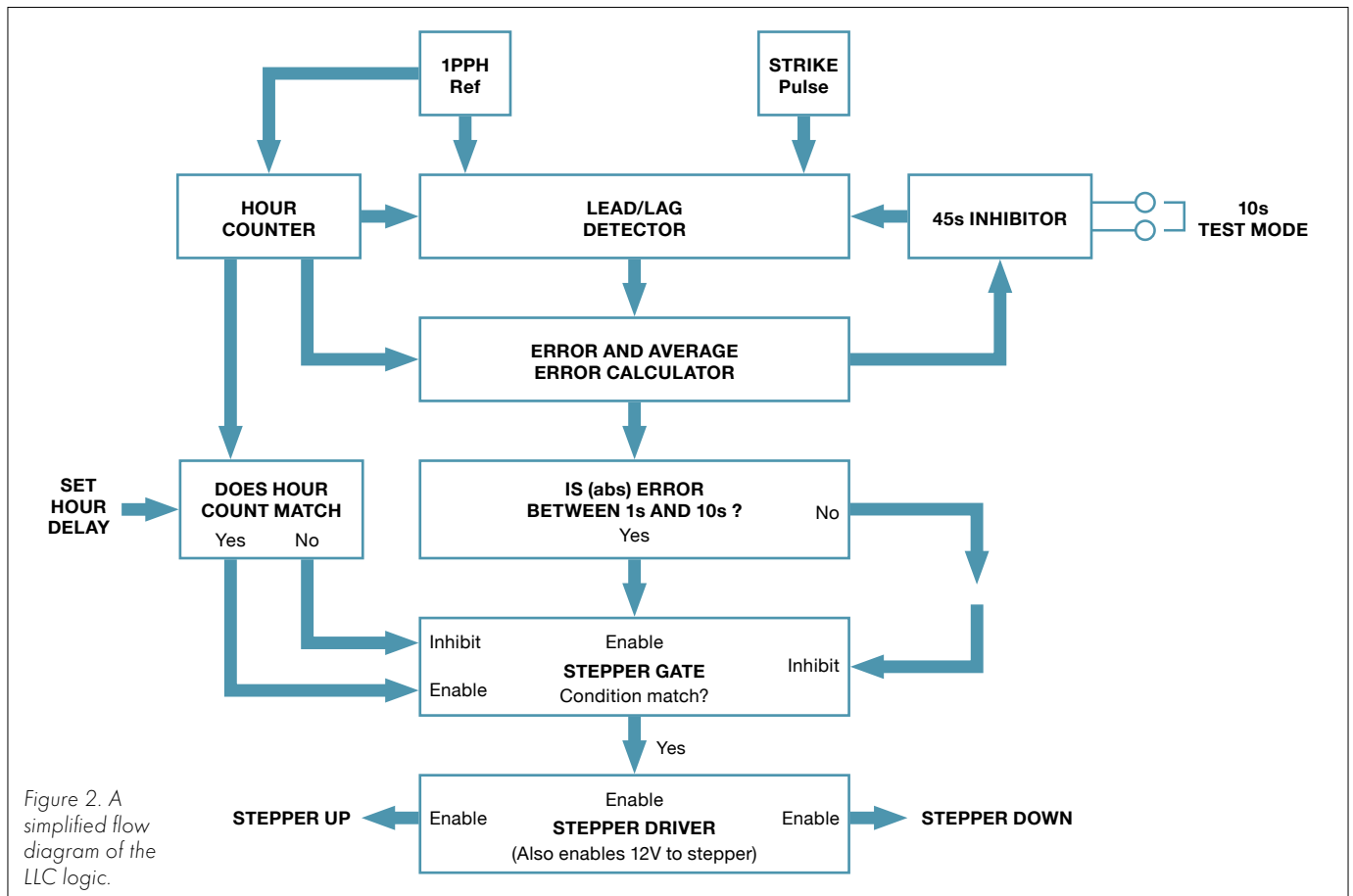
Being able to monitor the clock installation via the LoRa radio link gave a more detailed insight as to what was happening at the clock. This immediately brought some aspects to my attention:

1. Making the correction every hour did not match the dynamics of the clock. This led to overshoot of the compensation process. Variables were added to the LLC code to allow the hourly error value to be stored and averaged before a change could be signalled to the stepper motor. The number of hours used for the averaging was made programmable. If an error reading on any hour fell outside of desired limits, it was ignored and the hour averaging value adjusted.
2. Because of the physical position of the strike sensor, its trigger point was not at the same time as the sound of the church bell striking. A 'linkage' delay had to be added.
3. The strike sensor is triggered every hour and gives out a varied number of pulses dependent on the hour being struck. A blanking period was added to ignore subsequent hour strikes after the first one had sounded.

Resulting Software Detail

A simplified flow diagram of the LLC logic is shown in **Figure 2**. In summary:

1. Two external pulse sources are input to the lead lag logic controller (LLC) every hour on the hour. Each pulse triggers a rising edge interrupt to the Xiao processor.
2. The time difference between the two interrupt pulses is calculated to derive an error value.
3. While the error is measured every hour, there is a programmable counter that only lets stepper changes be made after a defined number of hours have passed. This delay allows the clock to stabilise after changes to the weights.
4. The error value for each passing hour is summed and stored and then divided by the relevant number of hours to derive an average error value over the period.
5. Follow-on pulses from either trigger source at the hour are ignored by an Inhibit period (45 s).



6. If the averaged error is within ± 1 s, the stepper is inhibited (clock is on time).
7. If an individual hour error is outside the ± 10 s window, the value is ignored and not added to the averaging process (likely to be a system failure).
8. If the subsequent averaged error also falls outside this ± 10 s window then the stepper is inhibited.
9. A fixed compensation delay is added to the strike arm sensor time stamp to compensate for the strike arm sensor trigger point and the actual bell sounding (circa 1 s).
10. There is an override link on the LLC PCB to provide a shorter inhibit period for testing purposes. The test period is defined in the code setup values.
11. Depending on whether the averaged error is a positive or negative value, is inside the 10 s window, and not in the 1 s 'on time' window, the stepper is driven to raise or lower the weight to the pendulum weight tray.
12. All the relevant setup values are adjustable in the program listing.

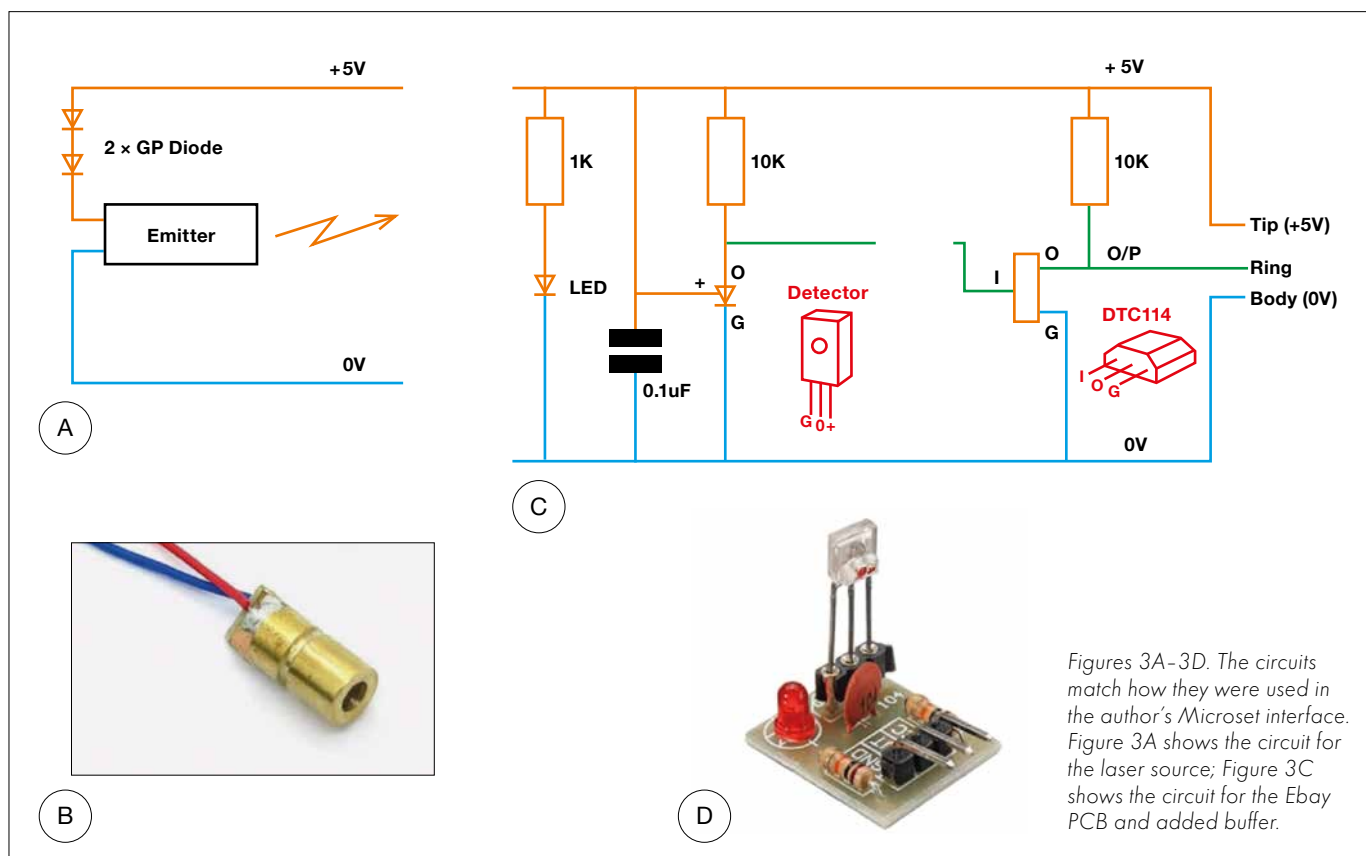
Final Hardware Implementation

The system required seven separate electronic modules, which I committed to printed circuit board designs. The boards were created using the electronics module in Fusion software. After the PCBs were proved they were manufactured offshore by PCBWay.

All Xiao processors were mounted in sockets, so code updates can be physically swapped out by changing the Xiao processors with newly coded ones hand carried from my workshop.

The installation also needed a number of 3D-printed hardware models to complement the electronics. The distinct individual items are listed below and more detailed descriptions follow:

1. Laser diode transmitter and receiver sensor module for the strike lever detector (3D printed mounting with hard wired interconnections) producing a 5 V logic level signal.
2. Xiao based GPS 1 pulse per hour reference pulse source (Xiao processor and GPS module and antenna) outputting a 5 V logic level signal.
3. Pulse detection lead lag logic controller (LLC) (Xiao processor).
4. Stepper control logic (Xiao processor with ULN2003 stepper driver module).
5. Power distribution module (simple PCB to distribute 12 V to all modules in the LLC).
6. Stepper motor mounting 3D printed hardware.
7. Stepper motor pulley (3D printed).
8. Weight tray extender wall (3D printed).
9. LoRa transmitter module (Xiao processor and RFM9x LoRa module).
10. LoRa receiver module (Xiao processor, RFM9x LoRa module, SD card storage, real time clock (RTC), SSD1327 OLED display).



Figures 3A–3D. The circuits match how they were used in the author's Microset interface. Figure 3A shows the circuit for the laser source; Figure 3C shows the circuit for the Ebay PCB and added buffer.

Where indicated, there is a circuit board design for the above modules. The figures in the description will show the Fusion Electronics rendering of the PCB component side view. This will be a bit more interesting than looking at my poor photographic skills. These images will also give a guide to the simplicity of the technology and ease of construction. Only through-hole components are used. Aside from the various Arduino modules, the remaining components are associated with diagnostic LEDs and decoupling. All components are mounted on the top side of the PCBs.

Anyone interested in just the clock controller can ignore the LoRa transmitter and LoRa receiver with no change needed to the LLC software code.

1. Strike Lever Sensor

The strike lever sensor is a U-shaped 3D print that contains a laser source and detector. This was originally constructed for use with my Microset timer. The laser emitter and detector can be bought on Ebay as a pair. The emitter is specified for 3V operation. This can be provided from a 5V supply via two diodes acting as a voltage dropper. The detector as supplied has a small PCB that powers the detector from a 5V supply. The output signal from the detector diode PCB needs buffering and inverting to give a positive going pulse to the LLC. A DTC114 digital transistor and a resistor can provide this. The circuits are as seen here and match how they were used in my Microset interface, **Figures 3A–3D**.

IMPORTANT NOTE: The laser emitter used in this detector has the potential to damage eyesight. Handle the device with caution.

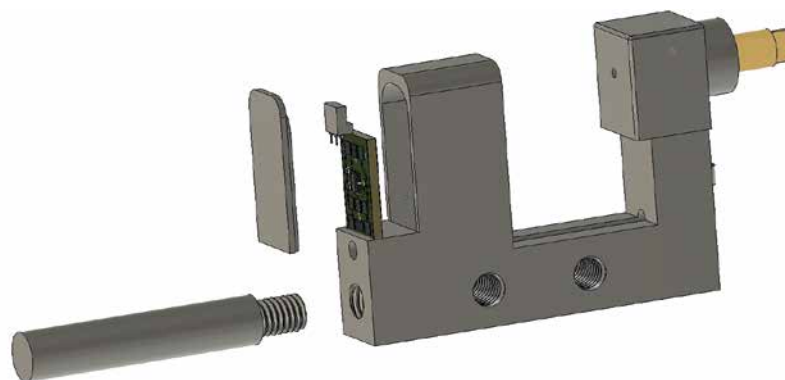


Figure 4. The U shaped 3D printed laser emitter and detector mounting module.

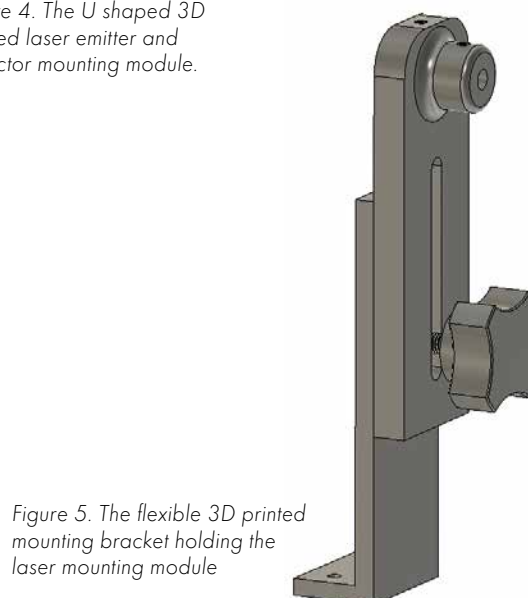


Figure 5. The flexible 3D printed mounting bracket holding the laser mounting module

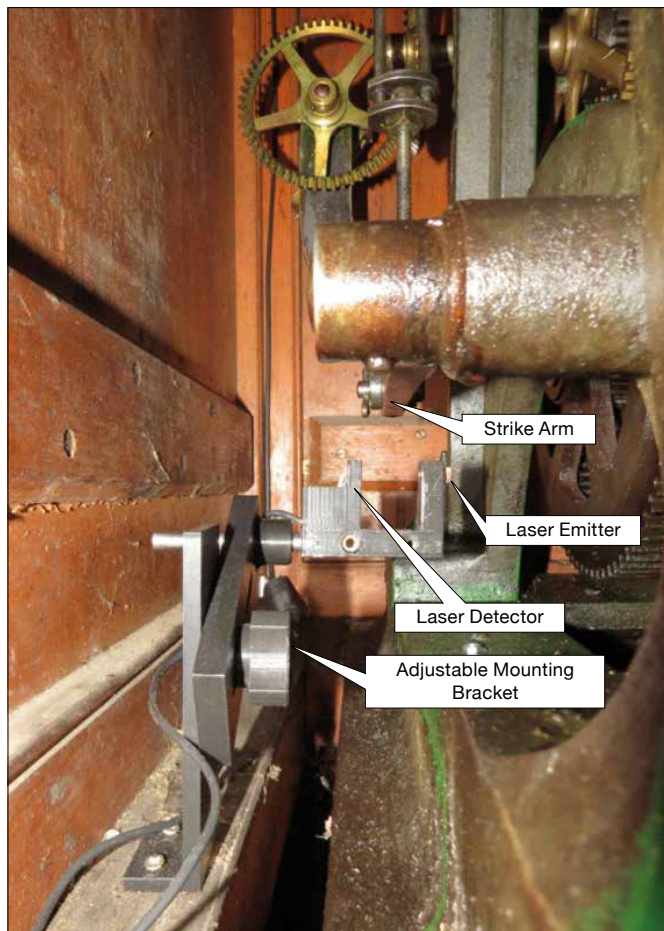


Figure 6. The complete laser detection assembly mounted at the rear of the clock.

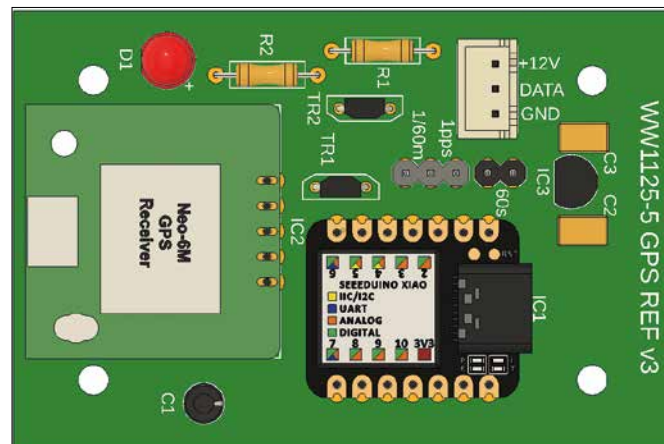
The 3D printed detector body is shown in **Figures 4 and 5**. The emitter and detector circuitry is mounted on a small surface mount PCB embedded in the 3D print with all wiring to and from each module encapsulated in channels in the print. The complete module incorporates a rod, which slides into an adjustable 3D printed bracket. The bracket is mounted at the rear of the clock cabinet. The construction is sufficiently flexible to allow simple adjustment of the beam trigger position relative to the drop of the strike arm. This concept will need adaption and customising to suit the mechanics of any other clock being controlled, **Figure 6**.

2. GPS 1PPH Pulse Source

The GPS reference source turned out to be simple from a hardware point of view, consisting of only a Xiao processor (IC1), an Arduino UBLOX GPS module (IC2) and a 12V to 5V regulator, **Figures 7 and 8**. The various modules are mounted on a PCB motherboard and housed inside a 3D-printed enclosure. The enclosure has a Perspex viewing window so the basic GPS 1 pulse per second LED and the divided 1 pulse per hour LED can be easily viewed.

The GPS housing is located adjacent to the LLC enclosure and connected via a three-core umbilical cable (+12V, GND and Data). The data signal is output from the Xiao as a 3V3 level. There is a simple circuit to buffer this to a 5V logic level. This helps noise immunity and matches the 5V TTL style legacy interface standard I had adopted on the LLC board.

For testing purposes there is a link on the PCB ('60s') to allow the 1pph pulse to be truncated to one per minute.



Figures 7 and 8. The GPS reference source, consisting of a Xiao processor (IC1), an Arduino UBLOX GPS module (IC2), and a 12V to 5V regulator.

The associated active GPS antenna is located in the belfry above the clock location. The antenna cable is fitted with a SMA male connector.

3. LLC Logic PCB

There is a plethora of diagnostic LEDs on the LLC board, **Figure 9**. These run down each side of the board to represent the two sensor chains. They indicate a sensor trigger, blanking period active and pulse out to the stepper (assuming conditions have been met).

The odd LED is LED (D4), which is driven by TR3. This shows the burst of data that is sent to the LoRa transmitter each hour. This is a high to low burst that is converted from a 3V3 standard to a 5V standard via TR1 and TR2. The data burst can also be monitored on an oscilloscope via the adjacent test point.

There are two 0.1 in pitch links on the board. One allows the blanking delay to be reduced when testing the system and is normally open circuit. The second link allows the Xiao processor to be powered down so any newly programmed device can be fitted without upsetting the rest of the

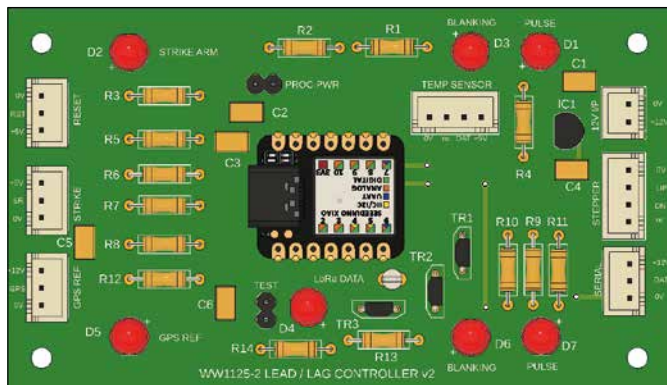


Figure 9. The LLC electronic controller PCB (97 mm x 57 mm).

electronics. This is particularly important to avoid disturbing the GPS reference, which takes time to stabilise if the power is disrupted.

4. Stepper Control PCB

The stepper controller is simple with only a Xiao processor, a ULN2003 stepper driver and a 5 V regulator, **Figure 10**. The Xiao receives a 100ms UP or DOWN pulse from the LLC PCB and drives the stepper motor either clockwise or anticlockwise to raise or lower weights onto the weight tray.

A series of four LEDs are included on the PCB, which are connected in parallel with the stepper drive signals from the ULN2003 (IC2). These allow visual checking of the stepper drive encoding. There is a link (JP1), which allows the stepper rotation to be selected as 180 degrees (link in place) or 90 degrees (link open circuit).

To reduce power consumption, the 12 V supply to the stepper is only switched ON (via relay K1) when a weight change is required.

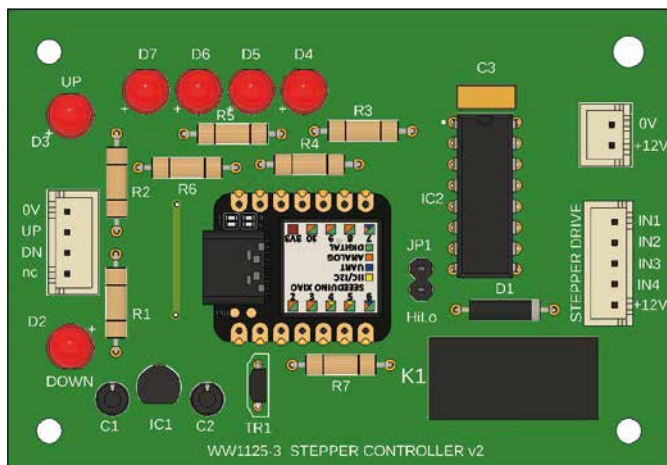


Figure 10. The Stepper controller PCB (80 mm x 57 mm).

5. Power Distribution

I opted to power all external and internal PCB modules from 12 V with a 5 V regulator on each board with sufficient capacity for the associated electronics. The 5 V rails supply the RAW input to the Xiao processors, which have their own integrated 3V3 regulators. This multi-tiered supply chain helps protect the various devices from cable induced transients and operator mistakes. The distribution of the 12 V within the LLC enclosure is done via a simple PCB. There is a surface mount Polyfuse on the rear side in the incoming 12 V supply.

6 Electronics Housing

The control electronics is housed in a standard off-the-shelf box with a transparent lid. This enables viewing of status indicators. The two controller boards (lead lag detector and stepper controller) are mounted side by side with the 12 V power distribution board mounted between them, **Figure 11**.

The majority of the interconnections within the enclosure and the other sub assembly PCBs use JST-XH crimp connectors, but the PCB footprints for this style of connectors will adapt to the use of 0.1 in SIL connectors or solder terminals.

Temperature is logged via a side process in the LLC Xiao. The temperature value is added to the LoRa radio link packet. The DHT22 temperature sensor is mounted external to the control box inside the church clock housing. The sensor is connected to the control box with a 3 core umbilical cable providing +5 V, Data and 0 V.

After fitting the LLC electronics into the box there was free space available. I used this area to add a separate temperature and humidity sensor PCB module. (This was available having been designed for a legacy project.) This uses a second DHT22 sensor mounted in the LLC box. This temperature reading is not integrated or reported over the LoRa radio link and has no bearing on the LLC system. It simply provides a useful local display for visitors to the clock.

The LLC enclosure also powers the original (but now redundant) off-air reference clock derived from DCF (Germany). This clock, like the independent temperature monitor, provides a useful visible reference while working local to the clock.

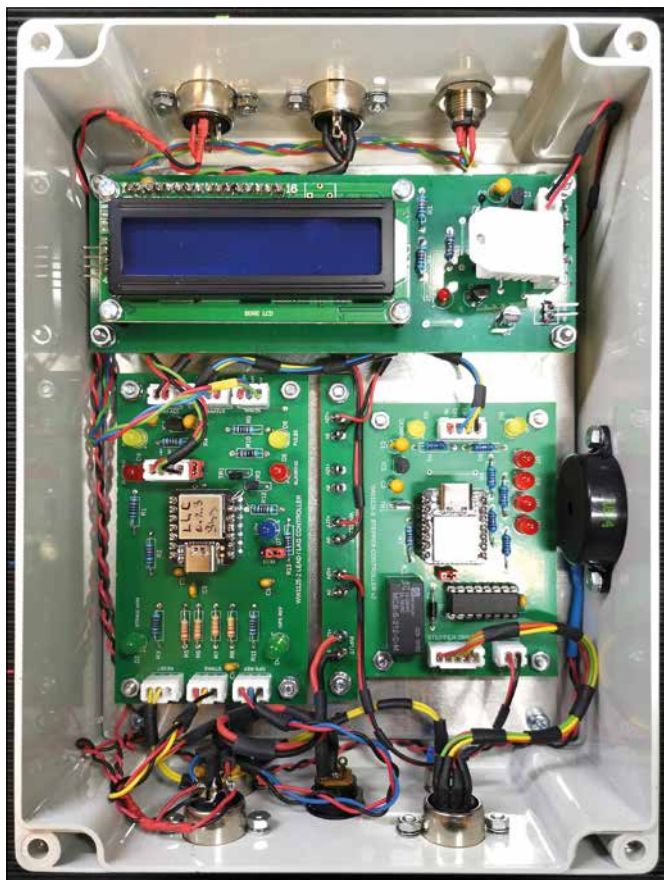


Figure 11. The internal assembly of the complete LLC enclosure. The power distribution PCB can be seen between the LLC PCB and the Stepper PCB. The display PCB at the top of the image is the legacy temperature and humidity monitor as mentioned in the text.

External connections to the LLC enclosure are via a mixture of XLR and Mini QG styles connectors. The connections are as follows:

1. Reference GPS 1 pph source input (XLR3).
2. Strike lever laser detector input (XLR3).
3. Stepper drive signal output (XLR5).
4. 12 V supply input (standard 2mm DC jack).
5. External DHT22 temperature sensor input (XLR3).
6. LoRa transmitter output (3 pin Mini QG).
7. Reset push button (push to activate).

The complete LLC enclosure is powered from a 12V 2A wall wart supply.

7. Stepper Mounting

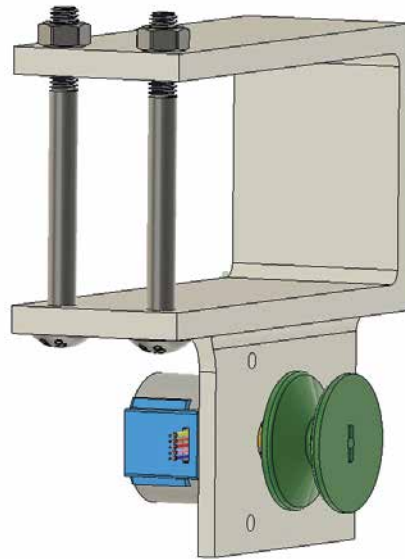
The stepper mounting was custom 3D printed to suit the clock and stepper geometry. It is clamped on the top beam of the clock frame. The stepper is a popular low-cost 12V item (28BYJ-48). **Figure 12** shows the Fusion pictorial view of the 3D mounting bracket and the 3D printed pulley. The geometry of the clamp allows sufficient adjustment to position the nylon line over the midpoint of the pendulum swing, **Figure 13**.

8. Weight Tray Ring

A 3D printed 'wall' extender ring was made to sit on the edge of the weight tray, **Figure 14**. This increases the depth of the weight tray to keep the dangling fishing weights within the tray.

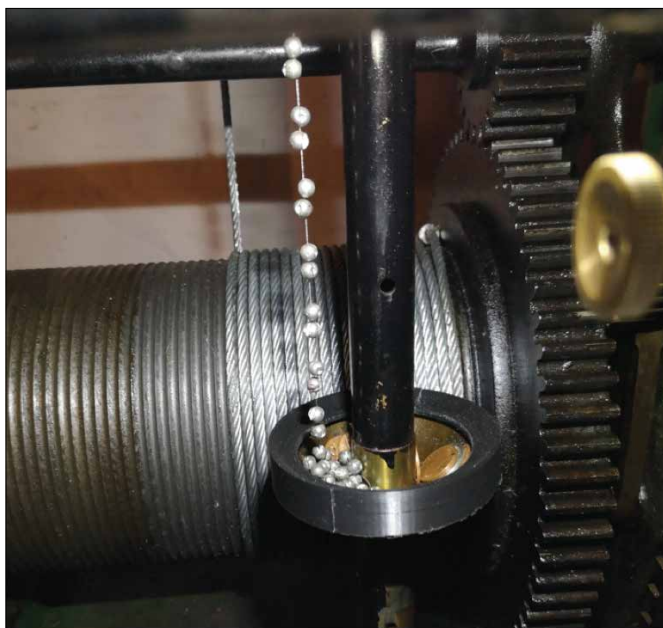
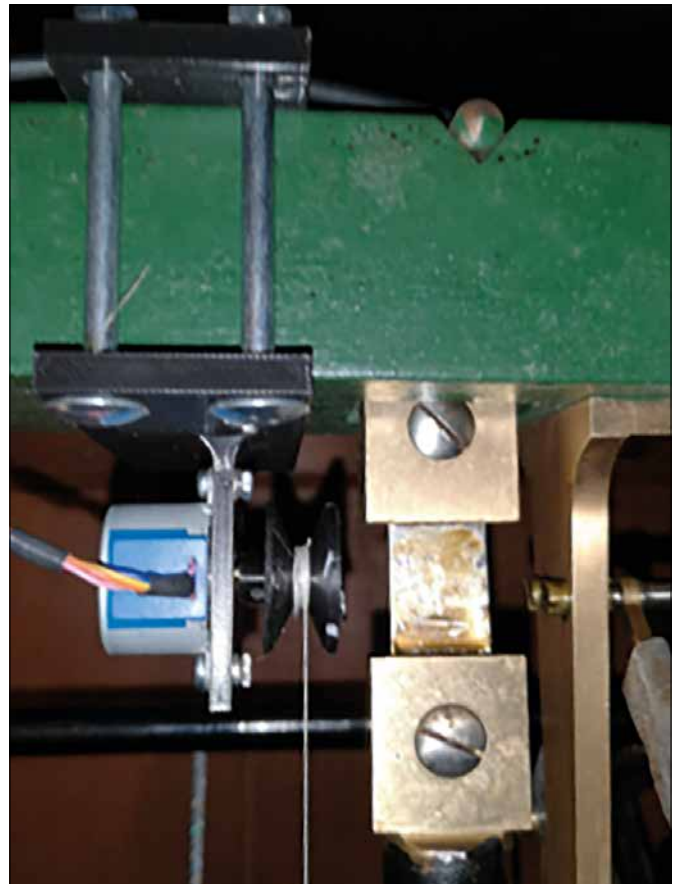
The ring was printed as two mirrored parts. These are clamped together around the pendulum rod using embedded 3mm diameter neodymium magnets. With care, the ring can then be gently dropped into place onto the tray without disturbing the clock. The circular sockets containing the magnets can be seen in the Fusion model.

The two half 3D prints of the wall and retaining magnets weighed around 20g, equivalent to six pennies being added to the tray. Sufficient loose change existed on the weight tray to allow compensation for this, **Figure 15**.



Figures 12. The 3D printed stepper mounting bracket and the 3D printed pulley wheel. The combination is seen mounted on the clock frame, Figure 13.

Figure 13. The geometry of the clamp allows sufficient adjustment to position the nylon line over the midpoint of the pendulum swing.



Figures 14. A 3D-printed 'wall' extender ring was made to sit on the edge of the weight tray.

Figure 15. Sufficient loose change existed on the weight tray to compensate for the change in weight.

9. LoRa Transmitter

The electronics for the LoRa transmitter consist of a Xiao (IC1), the LoRa module (IC2) and a 5V linear regulator, **Figures 16 and 17**.

The modules are mounted on a PCB and housed in a diecast enclosure. The transmitter box is mounted in the bell chamber above the clock. It is connected to the main LLC electronics box with an 8m long three-core cable (+12V, 0V and Serial data). The serial data from the LLC is at 5V logic level, which is resistively attenuated to 3V3 standard before feeding to the Xiao. There is a test pin (US\$1) to allow oscilloscope viewing of the data signal. (A normally high 5V level with the data pulsing low.) The antenna is an end fed dipole and is mounted on a SMA bulkhead connector directly on the end wall of the diecast box.

10. LoRa Receiver

The LoRa receiver is housed in a similar but smaller housing to the LLC main electronics with the same style clear plastic lid, **Figure 19**. The electronics for the receiver are almost exclusively made up of interconnected standard Arduino modules. These are LoRa receiver, Sseed Xiao processor, OLED display (SSD1327), real time clock (RTC) and microSD card module.

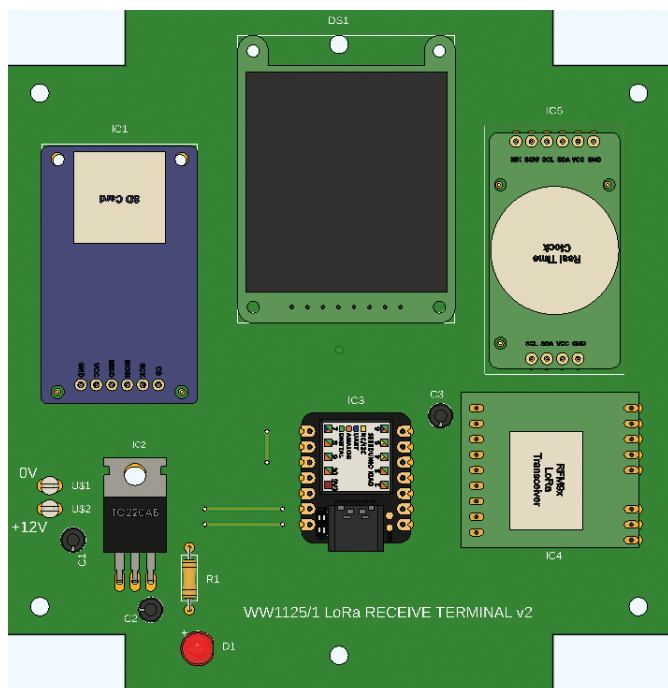
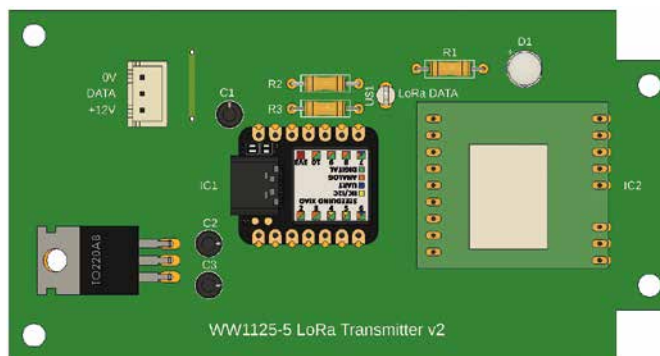


Figure 19. The LoRa receiver PCB (110 mm x 110 mm) is housed in a similar but smaller housing to the LLC main electronics with the same style clear plastic lid.

The modules are mounted on a PCB motherboard, which incorporates a 5V linear regulator. The complete receiver is powered via an external 12V source. The incoming received packet is locally time stamped using the real time clock module and the received signal strength is also added to the information displayed on the OLED screen. The data is also stored on the SD memory card in a CSV format (see **Figure 22**). As mentioned earlier, the incoming data packet includes the temperature in the church clock housing.

The SSD1327 OLED display auto scrolls through variously pre-formatted presentation screens showing the contents



Figures 16 and 17. The LoRa PCB (100 mm x 53 mm) with Xiao processor and LoRa receiver module housed in a diecast enclosure.

of the consolidated data packet received from the clock transmitter.

As with the LLC enclosure, the transparent lid allows viewing of the OLED monitoring screen and status LEDs. The LoRa receiver uses the same style end fed dipole as used on the transmitter. This is mounted on a SMA bulkhead connector on the end wall of the enclosure. **Figure 20** is an interconnect diagram of all the component parts of the system.

Installation

Figure 21 is the hardware adjacent to the clock with the production PCBs fitted. Left to right is the DCF off air clock, large LLC enclosure and the small GPS off air standard. The LoRa transmitter is in the bell room above the clock as is the GPS antenna. The production PCBs were all made offshore by PCBWay.

Setting Up

With the stepper disconnected from the LLC, a judgement was made on how many of the lead shot weights would be needed on the weight tray as a static starting point. This was mid-summer with the likelihood that the clock had reached its peak losing rate, due to temperature. As winter approaches

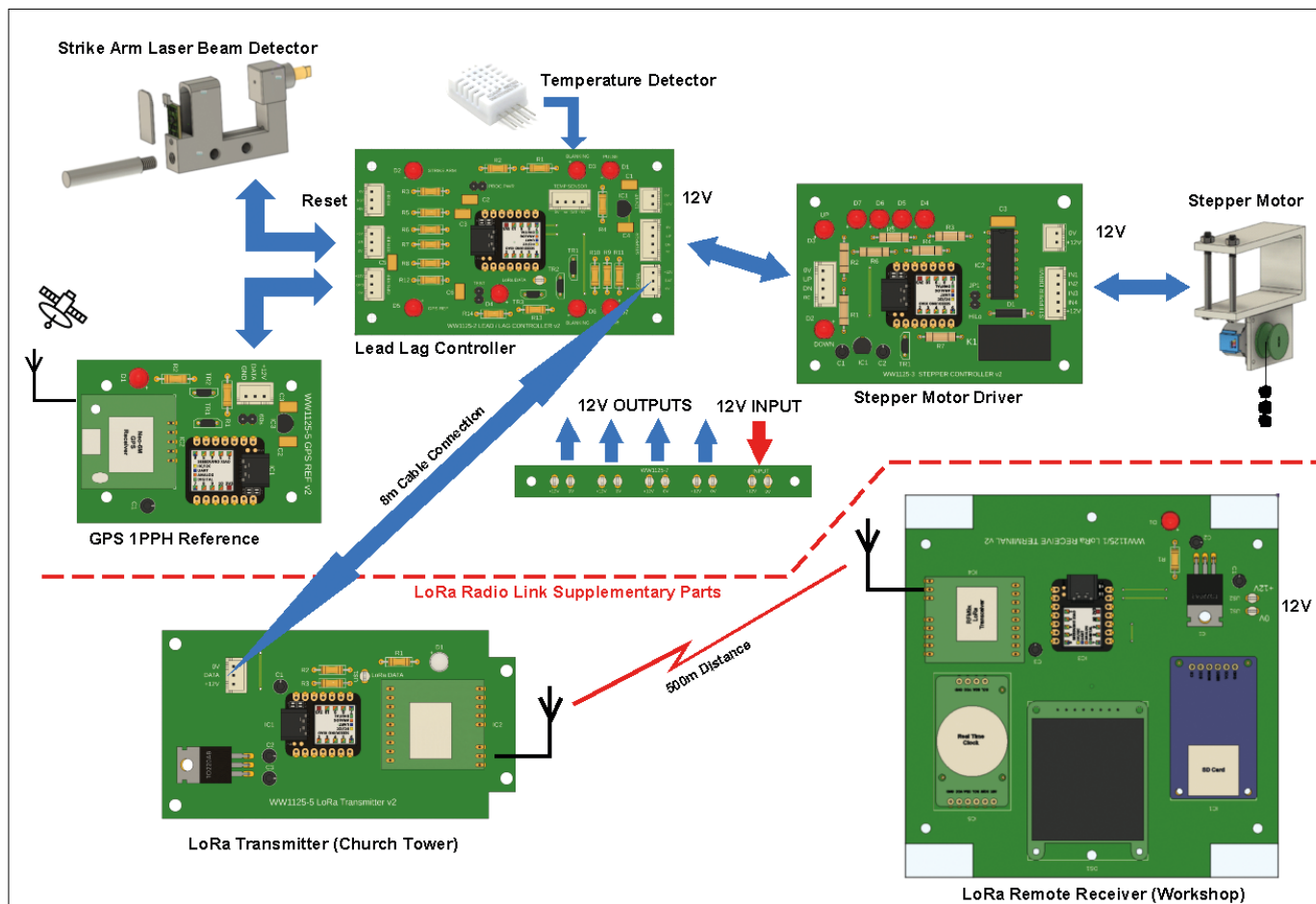


Figure 20. An interconnect diagram of all the component parts of the system.

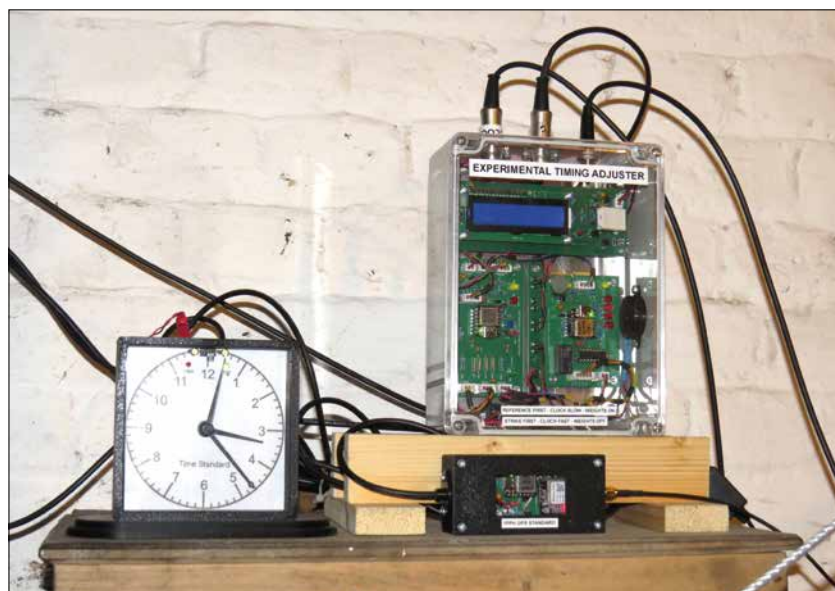


Figure 21. Left to right: the DCF off air clock, large LLC enclosure and the small GPS off air standard.

and temperatures cool there would be a speeding up of the clock needing the removal of weight from the tray to bring to time.

After I had removed sufficient pennies to compensate for my weight tray extender wall, I put the equivalent of two additional pennies (~7g) of weight on the tray as a starting

point and then manually monitored the clock over a few days. Some tweaking was necessary to get the starting balance between fishing weights and pennies.

With the clock appearing to have stabilised, the 1pph GPS reference clock was switched on and allowed to synchronise. The strike lever sensor was adjusted so that the laser beam sensor tripped as the chime initiated at the hour. This is best adjusted at 11am, with plenty of strikes to work with. If you get it wrong there is the noon strike to fall back on for another attempt.

The clock was monitored over the next few days to see the rate of change over time. From this I made a subjective decision on how often I wanted to enable a stepper motor change. I opted initially for a six-hour averaging period (this subsequently changed to a three-hour period).

The stepper was next connected enabling the full control loop. Automatic adjustment began to take place. These continued until the clock timing arrived within the ± 2 s blanking window before settling into a slow oscillation around the 2 s error value.

The remote display at my workshop has proved very useful to monitor all these variables. I am conscious that the sound of the chime at home is delayed by around 500ms and have to mentally add this to the displayed error values. **Figure 22** shows a typical CSV log file.

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Month	Day/Time	RSSI	Stike Clock	GPS Clock	Delta	Average	Hour	Valid	Strike Delta	GPS Delta	Temp	Action
2	1	10 09:59:33	-106	2226850600	2226851850	-1250	-1302	1	1	3601057	3601101	3	PAIR
3	1	10 10:59:33	-106	2230451865	2230452952	-1087	-1302	2	2	3601265	3601102	3	PAIR
4	1	10 11:59:33	-104	2234052947	2234054050	-1103	-1302	3	3	3601082	3601098	3	PAIR
5	1	10 12:59:33	-106	2237654197	2237655149	-952	-1146	1	1	3601250	3601099	2.9	PAIR
6	1	10 13:59:33	-106	2241255365	2241256248	-883	-1146	2	2	3601168	3601099	2.9	PAIR
7	1	10 14:59:33	-107	2244856545	2244857348	-803	-879	3	3	3601180	3601100	3	UP
8	1	10 15:59:33	-107	2248457695	2248458449	-754	-879	1	1	3601150	3601101	3	PAIR
9	1	10 16:59:33	-108	2252058852	2252059549	-697	-879	2	2	3601157	3601100	3.1	PAIR
10	1	10 17:59:33	-106	2255659992	2255660649	-657	-879	3	3	3601140	3601100	3.1	PAIR
11													
12	Month & Day/Time = Time stamp												
13	RSSI = Signal Strength of incoming LoRa radio transmission												
14	Strike Clock = Xiao internal clock millis count at Strike trigger												
15	GPS Clock = Xiao internal clock millis on GPS trigger												
16	Delta = Difference between Strike millis and GPS millis												
17	Average = Delta values averaged over hour count												
18	Hour = Hour count												
19	Valid = Number of Average values within an hour count that meet window criteria												
20	Strike Delta = Current Strike Clock millis less last Current Strike Clock millis												
21	GPS Delta = Current GPS Clock millis less last Current GPS Clock millis												
22	Temp = Current temperature inside the clock housing												
23	Action = Action taken on stepper where Pair is no action												

Figure 22. A typical CSV log file.

Conclusion

The short answer is that the LLC system works and is keeping the clock within my target tolerance; the software is stable and runs reliably. There is little point in documenting how well the synchroniser works as the aim of the project was to try to keep the clock within a second or so. This has been achieved. There is potential to improve on this with the existing hardware via code changes. However, in my opinion, this would be needlessly ‘gilding the lily’.

The residents of the village are unlikely to see any benefit from this advancement in accuracy. They will continue to do whatever they do, triggered by the sound of the church clock striking. They will see less of me walking back and forth to the church to add or remove pennies, but they will probably not realise the significance.

The project scratched an itch. It took me out of my comfort zone and made me dig deeper into my hobby interests in horology, software, electronics, 3D and CNC modelling, PCB design and 3D printing. It served as a useful introduction to ChatGPT, but that’s another story. I often wonder what my fellow Yorkshireman Thomas Cooke would have said had he been alive today...

A few observations about this interesting project were raised during review. While pennies or any weights added to the top of the pendulum will affect the clock's rate, it is not strictly their mass that creates this effect, but rather it is the position of said mass relative to the centre of oscillation. A tall thin object placed on top of the pendulum will affect its rate more than a wide flat one of equal mass. The position of an added mass relative to the overall length of the pendulum also makes a big difference – a mass added halfway along the distance between the suspension and the centre of oscillation will have a much greater effect than one placed further down. See the interesting article about this by Malcolm Pipes (HJ, October 2016, pp453–458).

Author's Note: Resource Availability

For readers who also have an itch to scratch, all the latest PCBs are available (please contact me via the email production@bhi.co.uk). Likewise, the Arduino source codes used and the Fusion models in STL format. Some knowledge of the technology involved might be a distinct advantage for anyone tempted to mimic my adventure. The physical arrangement of other clocks will be different and will need adaptation of the strike sensor and stepper mounting, but all other components will be directly applicable.

Author's Note: RF Propagation

This project, as documented, depends on two external ‘wireless’ functions, namely the GPS clock synchronising signal and the LoRa 868 MHz data monitoring link. Churches are traditionally built from thick stone material, and church towers often do not have many ‘radio’ apertures. These physical properties do not make the structure sympathetic to radio communications. This is something to think about and plan for during any similar installation.

Purists will also argue against having a tether (the string of fishing weights dangling above the pendulum) oscillating with the pendulum, although the effect is not likely to have a practical effect on such a heavy pendulum, especially with the author’s self-stated relaxed ‘village’ window of absolute timekeeping. Finally, one of the limitations when using opto-electronics as described is in the quality of the light detector. The ‘rise time’ of the signal produced by such a detector can vary enormously (ranging from milliseconds down to nanoseconds or less) depending on the detector quality and how it is configured. This is a consideration for those wanting to build such detectors into high-precision clocks.—HJ Editor.